

# Contents

|                                                                        |           |
|------------------------------------------------------------------------|-----------|
| <b>Drug Reviews — Module 3 Lab Guide</b>                               | <b>2</b>  |
| <b>The Scenario — Your Mission</b>                                     | <b>2</b>  |
| Where you work . . . . .                                               | 2         |
| The problem . . . . .                                                  | 3         |
| Your manager's request . . . . .                                       | 3         |
| Your team's job for the next 2 weeks (Module 3) . . . . .              | 3         |
| A note on ethics — read this carefully . . . . .                       | 3         |
| After Module 3 . . . . .                                               | 3         |
| <b>How This Guide Works (Read This First!)</b>                         | <b>4</b>  |
| We do NOT give you the full code . . . . .                             | 4         |
| <b>Visualizations — Two Modes</b>                                      | <b>4</b>  |
| 1. Exploratory charts (for YOU) . . . . .                              | 4         |
| 2. Explanatory charts (for DR. ANYA) . . . . .                         | 4         |
| Your plotting toolkit (you learned this in Module 2 Class 5) . . . . . | 5         |
| <b>Before You Start — Google Colab Setup</b>                           | <b>5</b>  |
| Step 1 — Open a new Colab notebook . . . . .                           | 5         |
| Step 2 — Connect to Google Drive . . . . .                             | 5         |
| Step 3 — Create a project folder . . . . .                             | 5         |
| Step 4 — Get the data . . . . .                                        | 5         |
| Step 5 — Test it . . . . .                                             | 6         |
| Colab tips . . . . .                                                   | 6         |
| <b>Class 1 — Data Cleaning</b>                                         | <b>6</b>  |
| Your goal . . . . .                                                    | 6         |
| Inputs . . . . .                                                       | 6         |
| Outputs . . . . .                                                      | 7         |
| Phase A — Explore the data first (15 minutes) . . . . .                | 7         |
| Phase B — Clean the two TSV files (45 minutes) . . . . .               | 7         |
| Phase C — Make ONE chart for Dr. Anya (15 minutes) . . . . .           | 9         |
| Common mistakes in Class 1 . . . . .                                   | 10        |
| Self-check before Class 2 . . . . .                                    | 10        |
| <b>Class 2 — Encoding and Scaling</b>                                  | <b>10</b> |
| Your goal . . . . .                                                    | 10        |
| Inputs . . . . .                                                       | 10        |
| Outputs . . . . .                                                      | 10        |
| Phase A — Explore the data first . . . . .                             | 10        |
| Phase B — Encode and scale . . . . .                                   | 11        |
| Phase C — Make ONE chart for Dr. Anya . . . . .                        | 12        |
| Common mistakes in Class 2 . . . . .                                   | 13        |
| Self-check before Class 3 . . . . .                                    | 13        |
| <b>Class 3 — Feature Engineering</b>                                   | <b>13</b> |
| Your goal . . . . .                                                    | 13        |
| Inputs . . . . .                                                       | 13        |
| Outputs . . . . .                                                      | 13        |
| Phase A — Explore the data first . . . . .                             | 14        |
| Phase B — Engineer the features . . . . .                              | 14        |
| Phase C — Make ONE chart for Dr. Anya . . . . .                        | 17        |
| Common mistakes in Class 3 . . . . .                                   | 17        |
| Self-check before Class 4 . . . . .                                    | 17        |

|                                                     |           |
|-----------------------------------------------------|-----------|
| <b>Class 4 — Feature Selection</b>                  | <b>17</b> |
| Your goal                                           | 17        |
| Inputs                                              | 18        |
| Outputs                                             | 18        |
| Phase A — Explore the data first                    | 18        |
| Phase B — Select features                           | 18        |
| Phase C — Make ONE chart for Dr. Anya               | 20        |
| Common mistakes in Class 4                          | 20        |
| Self-check before Class 5                           | 20        |
| <b>Class 5 — Pipelines</b>                          | <b>20</b> |
| Your goal                                           | 20        |
| Inputs                                              | 20        |
| Outputs                                             | 21        |
| Phase A — Explore the data first                    | 21        |
| Phase B — Build the pipeline                        | 21        |
| Phase C — Make ONE chart for Dr. Anya               | 23        |
| Common mistakes in Class 5                          | 23        |
| Self-check before Class 6                           | 23        |
| <b>Class 6 — End-to-End Lab (THE BIG ONE)</b>       | <b>23</b> |
| Your goal                                           | 24        |
| Time                                                | 24        |
| Inputs                                              | 24        |
| Outputs                                             | 24        |
| Phase A — Explore the data first (10 minutes)       | 24        |
| Phase B — Build the final dataset (70 minutes)      | 24        |
| Phase C — Findings report for Dr. Anya (15 minutes) | 25        |
| Grading rubric (100 points)                         | 26        |
| Submit                                              | 26        |
| <b>Tips for the whole module</b>                    | <b>27</b> |

## Drug Reviews — Module 3 Lab Guide

**Scenario:** UCI Drug Review Dataset (Drugs.com). Predict the patient rating (1 to 10) for a medication. **Module:** 3 (Data Preparation and Feature Engineering) **Audience:** Adult learners, A2 English. New to AI. **Tools:** Google Colab (no install needed).

## The Scenario — Your Mission

### Where you work

You and your team are new data analysts at **PharmaSignal**, a healthcare analytics company. PharmaSignal sells one product: reports about new medications. The customers are pharmaceutical companies, hospitals, and government health agencies.

Every day: - **5,000 new patient reviews** appear on websites like Drugs.com. - Each review has a star rating from 1 (terrible) to 10 (perfect). - Each review has free-text comments. Patients write about side effects, dose, doctor visits, and pregnancy.

## The problem

When a new medication is approved by the FDA, the FDA only sees data from **clinical trials**. A clinical trial has 500 to 5,000 people. They are watched for 6 months.

But **real-world use is different**: - 1 million people take the medication. - They use it for years, not months. - They mix it with other pills. - New side effects appear.

These real-world side effects show up in **patient reviews** on websites. But nobody at the pharma company reads 200,000 reviews. The information is lost.

## Your manager's request

Your manager is **Dr. Anya**, Director of Patient Insights at PharmaSignal. She is a medical doctor. She also studied data science. She tells you:

“When a new medication is approved, the FDA only sees clinical trials. Real-world side effects appear in patient reviews. I want a model that does two things.

One: **read patient reviews and predict the rating** (1 to 10). If the model is good, then the model UNDERSTANDS what makes a medication good or bad.

Two: **find words that signal side effects**. Which words appear in 1-star reviews but not in 10-star reviews? Words like ‘nausea’, ‘headache’, ‘dizziness’, ‘rash’.

Pharmaceutical companies will pay millions to know side effects 6 months earlier. Hospitals will pay to warn patients. Government agencies will pay to update safety labels.

Build me this model. Start with clean data.”

## Your team's job for the next 2 weeks (Module 3)

Dr. Anya cannot do this alone. The raw data is **2 messy TSV files** from the UCI Machine Learning Repository.

Your job in Module 3: > **Turn 2 messy TSV files into ONE clean file. The clean file will be used to train the model in Module 4.**

The clean file is called `drug_reviews_clean.parquet`. It must have **22 specific columns** (we will see them in Class 6).

## A note on ethics — read this carefully

This data comes from **real patients** who wrote about real medications. Even though the names are removed, this is still sensitive content.

- **Do NOT share raw patient text outside this lab.**
- **Do NOT post review text on social media** or in public notebooks.
- **Do NOT use this model to give medical advice.** Our model PREDICTS a rating. It does not tell anyone what to take.
- The dataset is published openly under the UCI license. That makes the data legal to use. But it does not make it polite. Treat each review as if it was written by your aunt.

Dr. Anya will check that you respect this.

## After Module 3

| Module          | What happens                                                            |
|-----------------|-------------------------------------------------------------------------|
| <b>Module 4</b> | Train the prediction model. Dr. Anya finally gets her rating predictor. |

| Module          | What happens                                                                       |
|-----------------|------------------------------------------------------------------------------------|
| <b>Module 5</b> | Find groups of patients (happy vs frustrated vs side-effect-driven). For research. |
| <b>Module 7</b> | Read the review text deeply. Find common side effect words. Find positive words.   |

You use the **same Drug Review dataset** until the end of Module 7.

## How This Guide Works (Read This First!)

### We do NOT give you the full code

In this guide you will see two kinds of content:

#### 1. WHAT / WHY / EXPECTED — explained in words

For every step you read: - **WHAT** you have to do (in normal language) - **WHY** you do it (the reason) - **HINTS** — the function names, the arguments to remember - **EXPECTED** — the result you should see if you did it right

#### 2. Sometimes a tiny code skeleton with blanks

For the harder steps we give you a code skeleton with \_\_\_\_ blanks. **You fill in the blanks.**

#### Why no full code?

Because you will NOT learn if you copy-paste. - In Module 4, the model only works if YOU understand each step. - In your future job, nobody will give you the code. - Writing the code yourself takes 10 minutes more today, but saves 10 hours later.

**Rule:** If you finish a step in 30 seconds by copy-pasting from a teammate's notebook, do it AGAIN by yourself. Different variable names, same logic.

## Visualizations — Two Modes

In every class you will make charts. There are **two reasons** to make a chart:

### 1. Exploratory charts (for YOU)

Made BEFORE you clean the data. To understand what the data looks like. - Fast, ugly, no labels needed. - Examples: `df.hist()`, `df['column'].value_counts().plot.bar()`.

Goal: “**What does this data look like?**”

### 2. Explanatory charts (for DR. ANYA)

Made AFTER you understand. To EXPLAIN something to your manager. - Clean, labeled, one clear message. - Must have: title, x-label, y-label, source, and a 1-sentence takeaway.

Goal: “**Dr. Anya, look at this. This is the medical signal.**”

In every class you make BOTH kinds.

## Your plotting toolkit (you learned this in Module 2 Class 5)

| Plot         | When to use it                         | Function name                                                |
|--------------|----------------------------------------|--------------------------------------------------------------|
| Histogram    | See the SHAPE of a numeric column      | <code>plt.hist()</code> or <code>sns.histplot()</code>       |
| Bar chart    | Count of each category                 | <code>df['col'].value_counts().plot.bar()</code>             |
| Box plot     | See outliers in a numeric column       | <code>sns.boxplot()</code>                                   |
| Scatter plot | See the relationship between 2 numbers | <code>plt.scatter()</code> or <code>sns.scatterplot()</code> |
| Heatmap      | See correlation between many columns   | <code>sns.heatmap(df.corr())</code>                          |
| Line plot    | See change over time                   | <code>plt.plot()</code>                                      |

Before you start: `import matplotlib.pyplot as plt` and `import seaborn as sns`.

## Before You Start — Google Colab Setup

You will work in **Google Colab**. That means: - You do NOT install Python. - You do NOT install pandas, scikit-learn, or anything. - You just open a notebook in your web browser.

### Step 1 — Open a new Colab notebook

1. Go to <https://colab.research.google.com>
2. Sign in with your Google account.
3. Click “**New notebook**” (top left).
4. Name it `drug_reviews_module_3.ipynb`.

### Step 2 — Connect to Google Drive

Colab files **disappear** when you close the browser. So you must save your work to **Google Drive**.

In the first cell:

```
from google.colab import drive
drive.mount('/content/drive')
```

A pop-up asks for permission. Click “Allow”. After this, Drive is available at `/content/drive/MyDrive/`.

### Step 3 — Create a project folder

In a new cell:

```
import os
os.makedirs('/content/drive/MyDrive/drug_reviews_lab', exist_ok=True)
%cd /content/drive/MyDrive/drug_reviews_lab
```

Your team will save ALL files in this folder.

### Step 4 — Get the data

#### Option A — Direct download from UCI (recommended):

The UCI Machine Learning Repository hosts the data. There are TWO files: a train file and a test file. Both are TSV (tab-separated, NOT comma-separated).

```
!wget -q https://archive.ics.uci.edu/static/public/462/drug+review+dataset+drugs+com.zip -O drug_r
!unzip -q -o drug_reviews.zip -d data
!ls data/
```

You should see two files: - drugsComTrain\_raw.tsv (about 161,000 rows) - drugsComTest\_raw.tsv (about 54,000 rows)

**Important:** This is **TSV**, not CSV. The separator is a TAB, not a comma. If you load it as CSV by mistake, everything will be in ONE column.

### Option B — Upload by hand:

1. Go to <https://archive.ics.uci.edu/dataset/462/drug+review+dataset+drugs+com> on your laptop.
2. Click “Download”.
3. Unzip on your laptop.
4. In Colab’s file panel (left sidebar), upload the two TSV files.

## Step 5 — Test it

```
import pandas as pd
train = pd.read_csv('data/drugsComTrain_raw.tsv', sep='\t')
print(train.shape)
print(train.columns.tolist())
```

Should print: - Shape: about (161297, 7). - Columns: ['uniqueID', 'drugName', 'condition', 'review', 'rating', 'date', 'usefulCount'].

If you see only 1 column, you forgot `sep='\t'`. Try again.

You are ready.

## Colab tips

| Tip                                                         | Why                                                  |
|-------------------------------------------------------------|------------------------------------------------------|
| Save your notebook to Drive often                           | Colab disconnects after 12 hours.                    |
| Save intermediate .parquet files to Drive                   | Files in /content/ disappear when Colab disconnects. |
| Share the notebook with teammates (Share button, top right) | Editor access. Like Google Docs for code.            |
| Restart runtime if memory full                              | Runtime > Restart runtime.                           |
| Do NOT print full review text in screenshots                | Privacy.                                             |

## Class 1 — Data Cleaning

**Scenario reminder:** Dr. Anya drops 2 messy TSV files on your desk. The file extension is .tsv. The date column is a string like “May 20, 2012”. Some condition fields are missing. Your job today: load both files, fix the dates, combine them, and find missing values.

### Your goal

Make the 2 TSV files **USABLE**. Load them with the right separator. Fix the date column. Combine train + test. Find missing values. Decide what to keep.

### Inputs

- The 2 TSV files in data/

## Outputs

- reviews\_step1.parquet saved in your Drive folder
  - 3+ exploratory charts in your notebook
  - 1 explanatory chart for Dr. Anya
  - Notes in markdown about every decision you made
- 

## Phase A — Explore the data first (15 minutes)

Before you clean anything, **LOOK** at the data.

### Exploratory chart 1 — How are ratings distributed?

- **Question:** “Of 215,000 reviews, how many give 1 star? How many give 10? Is it balanced?”
- **HINTS:**
  - Use `df['rating'].value_counts().sort_index()`.
  - Then `.plot.bar()` on the result.
- **What you learn:** The data is **heavily skewed toward 10 stars**. Many patients only write a review when very happy or very unhappy.

### Exploratory chart 2 — How long is a review?

- **Question:** “Are reviews short (1 sentence) or long (1 page)?”
- **HINTS:**
  - Make a column `review_length = df['review'].str.len()`.
  - Histogram with `bins=50`.
- **What you learn:** Most reviews are 100 to 500 characters. A few are over 2000.

### Exploratory chart 3 — Missing values per column

- **Question:** “Which columns have the most missing data?”
  - **HINTS:**
    - Use `df.isna().sum().sort_values(ascending=False)`.
    - Plot it as a bar chart.
  - **What you learn:** The condition column has missing values. The review column itself should not.
- 

## Phase B — Clean the two TSV files (45 minutes)

### Step 1 — Load both TSV files

- **WHAT:** Load `drugsComTrain_raw.tsv` and `drugsComTest_raw.tsv`. They are tab-separated.
- **HINTS:**
  - Use `pd.read_csv('data/drugsComTrain_raw.tsv', sep='\t')`.
  - Use `pd.read_csv('data/drugsComTest_raw.tsv', sep='\t')`.
  - **The separator is \t (tab), NOT comma.**
  - Give them short names: `train` and `test`.
- **EXPECTED:**
  - `train.shape` is about `(161297, 7)`.
  - `test.shape` is about `(53766, 7)`.
- **WHY load both?** UCI split the data for an old experiment. We do not care. We need ALL the reviews. We will split again later, our own way.

## Step 2 — Look at each DataFrame

- **WHAT:** For each, check `.shape`, `.info()`, and `.head()`.
- **HINTS:**
  - Loop over `[train, test]`. Or just call them one by one.
  - Look at the `Dtype` column in `.info()` output. **Is the date stored as object (text)?**
- **EXPECTED:**

| DataFrame | Approx rows | Columns |
|-----------|-------------|---------|
| train     | 161,297     | 7       |
| test      | 53,766      | 7       |

The 7 columns are: `uniqueID`, `drugName`, `condition`, `review`, `rating`, `date`, `usefulCount`.

## Step 3 — Combine train and test

- **WHAT:** Stack the two DataFrames on top of each other into ONE DataFrame called `df`.
- **HINTS:**
  - Use `pd.concat([train, test], ignore_index=True)`.
  - `ignore_index=True` makes the row index go 0, 1, 2, ... without duplicates.
- **WHY:** We have one project, one model. The old UCI split is not useful for us.
- **EXPECTED:** `df.shape` is about (215063, 7).

## Step 4 — Rename uniqueID to review\_id

- **WHAT:** Rename the column to a clearer name.
- **HINTS:**
  - `df = df.rename(columns={'uniqueID': 'review_id'})`.
- **WHY:** Our schema in Class 6 uses `review_id`. Match it early.

## Step 5 — Fix the date column

- **WHAT:** The date column is text like "May 20, 2012". Convert it to a real datetime.
- **HINTS:**
  - The function is `pd.to_datetime()`.
  - Add the argument `errors='coerce'`. If a cell is bad, it becomes `NaT` (Not a Time = missing). The code does not crash.
  - Example: `df['date'] = pd.to_datetime(df['date'], errors='coerce')`.
- **WHY:** If dates are strings, you cannot subtract them. "How old is this review?" is impossible without datetime.
- **EXPECTED:** After your code, `df['date'].dtype` is `datetime64[ns]`.

## Step 6 — Find missing values

- **WHAT:** Count missing values per column.
- **HINTS:** `.isna()` returns True/False per cell. `.sum()` counts the Trues per column.
- **EXPECTED:** Something like:

```
condition    ~1,200
review        0
rating        0
date          0
drugName      0
```



## Step 7 — Decide what to do with missing condition

- **WHAT:** About 1,200 rows have no condition. Three choices:
  - A. Drop these rows.
  - B. Fill with the string "Unknown".
  - C. Try to guess from the review text.
- **OUR CHOICE:** B (Fill with "Unknown"). Reason: we do not want to throw away patient data. "Unknown" is a real category.
- **HINTS:**
  - `df['condition'] = df['condition'].fillna('Unknown')`.

## Step 8 — Drop duplicate review IDs (just in case)

- **WHAT:** Keep only one row per review\_id.
- **HINTS:**
  - `df = df.drop_duplicates(subset='review_id')`.
- **WHY:** Sometimes the same review appears in both train and test files. We do not want it twice.

## Step 9 — Sanity check the rating column

- **WHAT:** Confirm rating is between 1 and 10 with no weird values.
- **HINTS:**
  - `df['rating'].describe()`. Min should be 1, max should be 10.
  - `df['rating'].value_counts().sort_index()` — there should be no 0 or 11.

## Step 10 — Write down what you did

In a markdown cell, write: - Train rows loaded: ~161,297 - Test rows loaded: ~53,766 - After combine: ~215,063 - After drop duplicates: ~215,000 - Missing condition: ~1,200 (filled with "Unknown") - WHY you made each decision.

## Step 11 — Save to Drive

- **WHAT:** Save the cleaned DataFrame as parquet, inside your Drive folder.
  - **HINTS:** `df.to_parquet('/content/drive/MyDrive/drug_reviews_lab/reviews_step1.parquet')`.
- 

## Phase C — Make ONE chart for Dr. Anya (15 minutes)

She does not have time to read your code. She wants ONE picture.

### Dr. Anya's chart — "Patients only review when very happy or very angry"

A bar chart showing the count of reviews for each rating, from 1 to 10.

- **HINTS:**
    - `df['rating'].value_counts().sort_index().plot.bar()`.
    - Add the title: "Rating distribution — patients write at the extremes".
    - X-label: "Rating (1=worst, 10=best)".
    - Y-label: "Number of reviews".
    - Color the bars: maybe red for 1-3, gray for 4-7, green for 8-10.
  - **Takeaway for Dr. Anya:** "About 60% of reviews are 8-10 stars. About 25% are 1-3 stars. Almost nobody writes a 5-star review. This is a U-shape, not a bell shape. Our model must handle this."
-

## Common mistakes in Class 1

| Mistake                                                         | What goes wrong                         |
|-----------------------------------------------------------------|-----------------------------------------|
| Use <code>pd.read_csv</code> without <code>sep='\t'</code>      | All 7 columns load as 1 column.         |
| Forget <code>errors='coerce'</code> on <code>to_datetime</code> | Code crashes on one bad date.           |
| Drop rows with missing condition without a reason               | You lose ~1,200 real reviews.           |
| Save to <code>/content/</code> instead of Drive                 | File disappears when Colab disconnects. |
| Print full review text to share                                 | Privacy violation.                      |

## Self-check before Class 2

- ☐ Train and test TSV loaded with `sep='\t'`.
- ☐ Combined into one DataFrame, about 215,000 rows.
- ☐ date column has dtype `datetime64`.
- ☐ condition has no NaN (filled with “Unknown”).
- ☐ At least 3 exploratory charts in your notebook.
- ☐ 1 polished chart for Dr. Anya.
- ☐ You saved `reviews_step1.parquet` to your Drive folder.
- ☐ You wrote down (in markdown) what you did and WHY.

## Class 2 — Encoding and Scaling

**Scenario reminder:** Dr. Anya looks at your cleaned data. She is happy. But she says: “the model is a math model. It does not understand the word ‘Depression’ or the drug name ‘Sertraline’. Turn the words into numbers. Also, `usefulCount` has a long tail — fix it.”

### Your goal

Turn TEXT columns into numbers. Make all numeric columns about the same size. Keep the raw review text — we need it in Module 7.

### Inputs

- `reviews_step1.parquet` from Class 1

### Outputs

- `reviews_step2.parquet` in Drive
- 3+ exploratory charts
- 1 explanatory chart for Dr. Anya

## Phase A — Explore the data first

### Exploratory chart 1 — Distribution of `usefulCount`

- **Question:** “Most reviews get 5 upvotes. A few get 1000+. Is the distribution skewed?”
- **HINTS:**
  - Use a histogram (`plt.hist()`).
  - Use `bins=50` to see the shape.
- **What you learn:** The `usefulCount` column has a “long tail”. This is why we will use log-transform.

## Exploratory chart 2 — Distribution of usefulCount AFTER np.log1p

- **Question:** “Does log-transform make the shape more normal?”
- **HINTS:**
  - Make a NEW column: `log_useful = np.log1p(df['usefulCount'])`.
  - Histogram it.
  - Compare to chart 1.
- **What you learn:** Log makes the long tail manageable for the model.

## Exploratory chart 3 — Top 20 conditions

- **Question:** “Which medical conditions appear most often?”
- **HINTS:**
  - `df['condition'].value_counts().head(20).plot.bar()`.
- **What you learn:** Birth Control, Depression, Pain, Anxiety, Acne, and Diabetes are the most common. The “long tail” of conditions has hundreds of rare diseases.

## Exploratory chart 4 — Top 20 drug names

- **Question:** “How many unique drugs are in the data? Which are most reviewed?”
  - **HINTS:**
    - `df['drugName'].nunique()` — how many unique values.
    - `df['drugName'].value_counts().head(20).plot.bar()`.
  - **What you learn:** About 3,500 unique drugs. Top 20 are common antidepressants and birth control pills. Long tail again.
- 

## Phase B — Encode and scale

### Step 1 — Bucket condition into top 10 + “Other”

- **WHAT:** condition has about 900 unique values. Too many. We will group them.
- **STRATEGY:**
  - Keep the top 10 most frequent conditions as their own buckets.
  - But also create **named clinical groups** so they make medical sense.
- **HINTS:**
  - First look at the top 10 with `df['condition'].value_counts().head(10)`.
  - Likely top 10 (your data may differ): Birth Control, Depression, Pain, Anxiety, Acne, Bipolar Disorder, Insomnia, Weight Loss, Obesity, ADHD.
  - Use Python dictionaries to map. Skeleton:

```
condition_buckets = {
    'Birth Control': 'Reproductive',
    'Depression': 'Mental Health',
    'Anxiety': 'Mental Health',
    'Bipolar Disorder': 'Mental Health',
    'Pain': 'Pain',
    'Insomnia': 'Sleep',
    'Acne': 'Skin',
    'Weight Loss': 'Weight',
    'Obesity': 'Weight',
    'ADHD': 'Mental Health',
}
```

`df['condition_top10_bucket'] = df['condition'].map(condition_buckets).fillna('__')`
  - Fill in the blank: anything not in the dictionary becomes 'Other'.
- **WHY this design?** Dr. Anya is a doctor. She wants groups that make medical sense. “Mental Health” tells a story. 900 separate conditions do not.
- **EXPECTED:** About 6 to 8 named buckets + “Other”.

## Step 2 — Target encode drugName

- **WHAT:** ~3,500 unique drugs. One-hot would create 3,500 columns. Too many. Use **target encoding**: replace the drug name with the average rating for that drug.
- **HINTS:**
  - Group by drugName. Take the mean of rating.
  - Make a Series drug\_avg.
  - Map back: `df['drugName_target_encoded'] = df['drugName'].map(drug_avg)`.
- **BIG WARNING:** This is **leakage** if you compute on the full data. Right now, we are still in cleaning. In Class 4 you will do this on TRAIN ONLY. For now, mark it as “preview only” in a comment.
- **WHY target encode and not one-hot?** Memory. Also, the model can use one number (“this drug averages 7.2 stars”) instead of 3,500 zero columns.

## Step 3 — Log-transform usefulCount

- **WHAT:** usefulCount has a very long tail. Apply `np.log1p()`.
- **HINTS:** `df['usefulCount_log'] = np.log1p(df['usefulCount'])`.
- **WHY log1p and not log?** `log(0)` is `-inf`. `log1p(x) = log(x + 1)` is safe for zeros. Many reviews have `usefulCount = 0`.

## Step 4 — One-hot encode condition\_top10\_bucket

- **WHAT:** Turn the 6-8 buckets into 6-8 new 0/1 columns.
- **HINTS:**
  - Use `pd.get_dummies(df, columns=['condition_top10_bucket'], prefix='cond')`.
- **EXPECTED:** New columns: `cond_Mental Health`, `cond_Pain`, `cond_Other`, etc.
- **NOTE:** We will keep the `condition_top10_bucket` text column AS WELL for the final schema. Make a copy before the one-hot if you need both.

## Step 5 — Scale numeric columns (preview only)

- **WHAT:** Use `StandardScaler` to make every numeric column have mean = 0, std = 1.
- **HINTS:**
  - `from sklearn.preprocessing import StandardScaler`.
  - `scaler = StandardScaler()`.
  - `scaler.fit_transform(df[numeric_cols])`.
- **WARNING:** This is for inspection only. In Class 5, scaling goes INSIDE the Pipeline. Do NOT save the scaled version as your final file.

## Step 6 — Save

- **WHAT:** Save to Drive. `df.to_parquet('/content/drive/MyDrive/drug_reviews_lab/reviews_step2.parquet')`
- **Save the UNSCALED version.** Scaling will happen inside the Pipeline later.

---

## Phase C — Make ONE chart for Dr. Anya

### Dr. Anya’s chart — “Rating depends on the condition”

A grouped bar chart of average rating per `condition_top10_bucket`.

- **HINTS:**
  - `df.groupby('condition_top10_bucket')['rating'].mean().sort_values()`.
  - Then `.plot.barh()`.
- **Title:** “Average rating by medical area – pain meds rate lowest, contraceptives rate highest”.
- **X-label:** “Average rating (1–10)”.

- **Y-label:** "Medical area".
- **Takeaway:** "Depression and pain medications get the lowest scores. Birth control gets the highest. The condition is a strong signal. Our model must use it."

---

## Common mistakes in Class 2

| Mistake                                                           | What goes wrong                                                               |
|-------------------------------------------------------------------|-------------------------------------------------------------------------------|
| Scale BEFORE train/test split                                     | <b>Leakage.</b> Scaler sees test data.                                        |
| One-hot encode drugName (3,500 unique)                            | Table explodes. Memory crash.                                                 |
| Forget <code>np.log1p</code> and use <code>np.log</code> on zeros | <code>np.log(0) = -inf</code> . Crash.                                        |
| Target encode using FULL data                                     | Leakage (fix in Class 4).                                                     |
| Save the scaled version                                           | Class 5 will scale again inside the pipeline. Double scaling = wrong numbers. |

---

## Self-check before Class 3

- ☐ `condition_top10_bucket` exists with about 7 named groups.
  - ☐ `drugName_target_encoded` exists (preview).
  - ☐ `usefulCount_log` exists.
  - ☐ One-hot columns for the condition bucket exist.
  - ☐ At least 3 exploratory charts.
  - ☐ 1 chart for Dr. Anya.
  - ☐ `reviews_step2.parquet` saved to Drive.
- 

## Class 3 — Feature Engineering

**Scenario reminder:** Dr. Anya says: "The raw columns are not enough. The signal is INSIDE the review text. Words like 'nausea' and 'dizziness' predict 1-star ratings. Words like 'helped' predict 10-star ratings. Build me features that COUNT these words."

### Your goal

Make NEW columns from the existing ones. The biggest new columns will come from the **review text**. We will use simple keyword counts (rule-based features), NOT machine learning yet. ML on text comes in Module 7.

### Inputs

- `reviews_step2.parquet`

### Outputs

- `reviews_step3.parquet` in Drive
  - 3+ exploratory charts
  - 1 explanatory chart for Dr. Anya
-

## Phase A — Explore the data first

### Exploratory chart 1 — is\_effective distribution

- First make the `is_effective` column (see Step 1 below).
- **Question:** “What % of reviews are ‘effective’ (rating  $\geq 7$ )?”
- **HINTS:** `df['is_effective'].value_counts(normalize=True).plot.bar()`.
- **What you learn:** Around 65-70% of reviews are “effective”. The classes are not balanced (we will handle this in Class 5).

### Exploratory chart 2 — Review length vs rating

- **Question:** “Do angry patients write longer reviews than happy patients?”
- **HINTS:**
  - Make `review_length` (Step 3 below).
  - Use `df.groupby('rating')['review_length'].mean()`.
  - Plot as a bar chart.
- **What you learn:** Often, the most extreme reviews (1 and 10 stars) are the longest. People with strong feelings write more.

### Exploratory chart 3 — Side-effect keywords frequency

- After you compute `n_side_effect_keywords` (Step 5), look at its distribution.
- **HINTS:** Histogram with `bins=20`.
- **What you learn:** Most reviews have 0 side-effect keywords. A few reviews mention 5+ different symptoms. These are the gold mine.

### Exploratory chart 4 — Side-effect count vs rating

- **Question:** “Are reviews with many side-effect words rated lower?”
  - **HINTS:**
    - Group `n_side_effect_keywords` (or `pd.cut()` it into 5 bins).
    - Compute mean rating per bin.
    - Bar chart.
  - **What you learn:** This is the most important chart. Side-effect words strongly predict low ratings.
- 

## Phase B — Engineer the features

### Step 1 — Create the engineered binary target `is_effective`

- **WHAT:** `is_effective = 1` if rating  $\geq 7$ , else 0.
- **HINTS:**
  - Compare: `df['rating'] >= 7`.
  - The result is True/False. Convert to int with `.astype(int)`.
- **EXPECTED:** About 65-70% of rows have `is_effective = 1`. Confirm with `df['is_effective'].mean()`.
- **WHY both rating AND is\_effective?** In Module 4, you will try BOTH problems: regression (predict the exact number) AND classification (predict effective yes/no). Keep both columns.

### Step 2 — Date-derived features

Make these new columns from the date column:

| New column                | What it is           |
|---------------------------|----------------------|
| <code>review_year</code>  | The year (2008-2017) |
| <code>review_month</code> | The month (1-12)     |

| New column       | What it is                             |
|------------------|----------------------------------------|
| review_age_years | Years from review date to today (2026) |

- **HINTS:**
  - Use the `.dt` accessor on a datetime column.
  - `df['date'].dt.year, .dt.month.`
  - For `review_age_years`:  
`today = pd.Timestamp('2026-05-21')`  
`df['review_age_years'] = (today - df['date']).dt.days / 365.25`
- **WHY:** A model can learn “old reviews are less reliable” only if you give it the age.

### Step 3 — Text-shape features

Simple counts ABOUT the review (not the meaning). These work in any language.

| New column    | What it is                | How to compute                                                      |
|---------------|---------------------------|---------------------------------------------------------------------|
| review_length | Number of characters      | <code>df['review'].str.len()</code>                                 |
| n_sentences   | Number of sentences       | Split by <code>.</code> , <code>!</code> , <code>?</code> and count |
| n_caps_words  | Number of UPPERCASE words | Count words that are all caps                                       |

- **HINTS for n\_sentences:**
  - Use `df['review'].str.count(r'[.!?]')`.
  - Sentences end with `.`, `!`, or `?`.
- **HINTS for n\_caps\_words:**
  - You need to split the review into words.
  - Then count how many words are all uppercase AND have at least 2 letters (to avoid counting “I” or “A”).
  - Skeleton:  

```
def count_caps(text):
    if not isinstance(text, str):
        return 0
    words = text.split()
    caps = [w for w in words if w.isupper() and len(w) >= ____]
    return len(caps)
df['n_caps_words'] = df['review'].apply(____)
```
  - Fill in the blanks.
- **WHY caps?** “TERRIBLE”, “AWFUL”, “NEVER AGAIN” — angry patients SHOUT in text.

### Step 4 — Define your keyword dictionaries

You will make THREE word lists. Then count appearances per review.

```
side_effect_words = [
    'nausea', 'headache', 'dizziness', 'dizzy', 'vomit', 'vomiting',
    'rash', 'itching', 'drowsy', 'drowsiness', 'fatigue', 'tired',
    'insomnia', 'anxiety', 'depression', 'weight gain', 'weight loss',
    'diarrhea', 'constipation', 'dry mouth', 'sweating', 'tremor',
    'pain', 'cramps', 'bleeding', 'swelling',
]

positive_words = [
    'helped', 'great', 'amazing', 'love', 'wonderful', 'excellent',
    'works', 'effective', 'recommend', 'happy', 'relief', 'better',
    'saved', 'lifesaver', 'perfect',
]
```

```
negative_words = [
    'terrible', 'useless', 'awful', 'horrible', 'worst', 'waste',
    'never again', 'do not', 'stopped', 'quit', 'switched',
    'allergic', 'reaction', 'emergency', 'hospital',
]
```

You can extend these lists. Write your own. Show Dr. Anya the list and let her add medical terms.

### Step 5 — Count keywords per review

- **WHAT:** For each review, count how many keywords from each list appear.
- **HINTS:**

- Lowercase the review first.
- Then for each word in the list, count appearances.
- Skeleton:

```
def count_keywords(text, words):
    if not isinstance(text, str):
        return 0
    text_lower = text.__()
    total = 0
    for w in words:
        total += text_lower.count(____)
    return total
```

```
df['n_side_effect_keywords'] = df['review'].apply(lambda t: count_keywords(t, ____))
df['n_positive_keywords'] = df['review'].apply(lambda t: count_keywords(t, ____))
df['n_negative_keywords'] = df['review'].apply(lambda t: count_keywords(t, ____))
```

- Fill in the blanks.

- **WHY count by hand instead of ML?** This is a SIMPLE, transparent feature. Dr. Anya can EXPLAIN it to the FDA. “Our model counted the word ‘nausea’.” That is trustable. ML black boxes are not.

### Step 6 — Three more rule-based flags

| New column         | What it is                                                 | How        |
|--------------------|------------------------------------------------------------|------------|
| has_specific_dose  | 1 if the review mentions a number+unit like “20 mg”        | Regex      |
| mentions_doctor    | 1 if “doctor”, “dr.”, “physician”, “GP” appears            | Word check |
| mentions_pregnancy | 1 if “pregnant”, “pregnancy”, “trying to conceive” appears | Word check |

- **HINTS for has\_specific\_dose:**
  - Use a regex: `r'\d+\s*(mg|ml|mcg|g|iu)'`.
  - `df['has_specific_dose'] = df['review'].str.lower().str.contains(r'\d+\s*(mg|ml|mcg|g|iu)', regex=True, na=False).astype(int)`.
- **HINTS for the other two:**
  - Use `.str.lower().str.contains('doctor|physician|gp', regex=True, na=False).astype(int)`.
- **WHY these features?**
  - has\_specific\_dose: shows the review is detailed and credible.
  - mentions\_doctor: shows the patient followed medical guidance.
  - mentions\_pregnancy: critical for safety analysis. Pharma companies care a LOT about pregnancy-related reviews.

### Step 7 — Save

- **HINTS:** `df.to_parquet('/content/drive/MyDrive/drug_reviews_lab/reviews_step3.parquet')`.



---

## Phase C — Make ONE chart for Dr. Anya

### Dr. Anya's chart — "Side-effect keywords predict low ratings"

A bar chart showing: bin `n_side_effect_keywords` into 5 buckets, show the **mean rating** in each bucket.

- **HINTS:**
    - `pd.cut(df['n_side_effect_keywords'], bins=[-0.1, 0, 1, 3, 5, 50]).`
    - GroupBy that, take the mean of rating.
    - Bar chart.
  - **Title:** "Average rating drops as side-effect words appear".
  - **X-label:** "Number of side-effect keywords in the review".
  - **Y-label:** "Average rating (1-10)".
  - **Takeaway for Dr. Anya:** "Reviews with zero side-effect words average 8 stars. Reviews with 5+ side-effect words average 3 stars. Counting these words alone tells us a lot. If a new drug shows a sudden spike in side-effect word counts, that is an early warning."
- 

## Common mistakes in Class 3

| Mistake                                                     | What goes wrong                                                                  |
|-------------------------------------------------------------|----------------------------------------------------------------------------------|
| Forget to lowercase before keyword count                    | "Nausea" is counted but "NAUSEA" is not.                                         |
| Use <code>delivery_days</code> -style "future" features     | (Not applicable here, but never use a column that depends on the rating itself.) |
| Forget <code>na=False</code> on <code>str.contains</code>   | NaN values become True. Wrong counts.                                            |
| Use the count of one word ("pain") that is ALSO a condition | "Pain" is both a side effect AND a condition name. Be careful.                   |
| Make the keyword list too small                             | If your list has only 3 words, the feature is weak.                              |

## Self-check before Class 4

- ☐ `is_effective` exists. Mean ~65-70%.
  - ☐ `review_year`, `review_month`, `review_age_years` exist.
  - ☐ `review_length`, `n_sentences`, `n_caps_words` exist.
  - ☐ `n_side_effect_keywords`, `n_positive_keywords`, `n_negative_keywords` exist.
  - ☐ `has_specific_dose`, `mentions_doctor`, `mentions_pregnancy` exist.
  - ☐ 3+ exploratory charts.
  - ☐ 1 explanatory chart.
  - ☐ `reviews_step3`.parquet saved to Drive.
- 

## Class 4 — Feature Selection

**Scenario reminder:** Now you have ~22 columns. Dr. Anya says: "Too many. Some are duplicates. Some are useless. Pick the ones the model actually needs. And remember: I want a model I can EXPLAIN to the FDA."

### Your goal

Pick the best columns. Drop the useless ones. Justify every choice. Remember: in Module 4 you might do regression (predict rating) OR classification (predict `is_effective`). Your selection should work for both.

## Inputs

- reviews\_step3.parquet

## Outputs

- reviews\_step4.parquet in Drive (only the selected columns + rating + is\_effective)
  - 3+ exploratory charts (correlation heatmap, mutual info bars, importance bars)
  - A short markdown report explaining your choices
- 

## Phase A — Explore the data first

### Exploratory chart 1 — Correlation heatmap of numeric columns

- **Question:** “Which columns say the same thing as each other?”
- **HINTS:**
  - Compute `df[numeric_cols].corr()`.
  - Plot with `sns.heatmap()`.
  - Use `annot=True` and `fmt='.2f'`.
- **What you learn:** Pairs of columns with `lcorr > 0.9` are redundant. For example, `review_length` and `n_sentences` are probably highly correlated.

### Exploratory chart 2 — Mutual information bar chart

- After you compute mutual info (Step 4 below), plot it.
- **HINTS:**
  - Sort the values, then bar chart.
- **What you learn:** Which columns predict rating (or `is_effective`) strongest.

### Exploratory chart 3 — Random Forest feature importance

- After you train the RF (Step 5 below), plot the importances.
  - **HINTS:** `pd.Series(rf.feature_importances_, index=...).sort_values().plot.barh()`.
  - **What you learn:** A second opinion on feature importance.
- 

## Phase B — Select features

### Step 1 — Split into train and test FIRST

- **WHAT:** Use `train_test_split`.
- **HINTS:**
  - from `sklearn.model_selection` import `train_test_split`.
  - `X = df.drop(['rating', 'is_effective', 'review'], axis=1)`.
  - `y = df['is_effective']`.
  - Pass: `test_size=0.2, random_state=42, stratify=y`.
- **WHY drop review from X?** It is text. We will use it in Module 7, not now. But we save it in the final file.
- **WHY split FIRST?** All next steps (target encoding, mutual info) must use TRAIN ONLY. Otherwise leakage.

### Step 2 — Recompute target encoding on TRAIN ONLY

- **WHAT:** In Class 2 you did `drugName_target_encoded` with the full data. That was leakage. Fix it now.
- **HINTS:**
  - `drug_avg_train = X_train.groupby('drugName')['rating'].mean()` — wait, rating is not in X. So use `y_train` indirectly. Skeleton:

```

train_temp = X_train.copy()
train_temp['rating'] = df.loc[X_train.index, 'rating']
drug_avg_train = train_temp.groupby('___')['___'].mean()
X_train['drugName_target_encoded'] = X_train['drugName'].map(drug_avg_train)
X_test['drugName_target_encoded'] = X_test['drugName'].map(drug_avg_train) # use TRAIN m

```

- Fill in the blanks.
- For drugs in test that were not in train, the result will be NaN. Fill with the global train mean.
- **WHY:** This is the core idea of preventing data leakage. The test set must look like “new data” to the model.

### Step 3 — Remove low-variance columns

- **WHAT:** Drop columns where almost all values are the same.
- **HINTS:**
  - from sklearn.feature\_selection import VarianceThreshold.
  - VarianceThreshold(threshold=0.01) then .fit\_transform(X\_train[numeric\_cols]).

### Step 4 — Remove highly correlated columns

- **WHAT:** Drop one of each pair where  $|corrl| > 0.9$ .
- **HINTS:**
  - Compute correlation matrix.
  - Get the upper triangle with `np.triu(...)`.
  - For each column, if any of its upper-triangle correlations is  $> 0.9$ , mark it for dropping.
- **EXPECTED:** Likely review\_length and n\_sentences are too similar. Keep one.

### Step 5 — Rank by mutual information

- **WHAT:** Score each column by how much it tells you about is\_effective.
- **HINTS:**
  - from sklearn.feature\_selection import mutual\_info\_classif.
  - `mi = mutual_info_classif(X_train_numeric, y_train)`.
  - Put in a Series, sort.
- **EXPECTED:** n\_side\_effect\_keywords, n\_negative\_keywords, n\_positive\_keywords, drug-Name\_target\_encoded, condition\_top10\_bucket should be at the top.

### Step 6 — Random Forest importance (second opinion)

- **WHAT:** Train a small RF, look at feature\_importances\_.
- **HINTS:**
  - from sklearn.ensemble import RandomForestClassifier.
  - RandomForestClassifier(n\_estimators=50, max\_depth=8, n\_jobs=-1, random\_state=42).
  - .fit(X\_train, y\_train).
  - Look at .feature\_importances\_.

### Step 7 — Pick the final columns

- **WHAT:** Combine the rankings. Pick columns that:
  - Are high in mutual info, AND
  - Are high in RF importance, AND
  - Were not dropped by variance/correlation pruning.
- **WRITE DOWN:** In your notebook, list the columns. Explain in 1 sentence why each one is in.

### Step 8 — Save

- **HINTS:** Keep only the selected columns + rating + is\_effective + review (for Module 7). Save as re-views\_step4.parquet to Drive.

---

## Phase C — Make ONE chart for Dr. Anya

### Dr. Anya's chart — "These are the words that predict effectiveness"

A horizontal bar chart of your top 10 features and their importance score.

- **HINTS:**
    - Use the RF importance.
    - `sort_values()` then `.head(10)` then `.plot.barh()`.
  - **Title:** "Top 10 predictors of effective rating".
  - **Y-axis:** column names.
  - **X-axis:** Random Forest importance.
  - **Takeaway:** "Side-effect word count and the drug's average historical rating predict 60% of the signal. Everything else is small. We can build a transparent, simple model."
- 

## Common mistakes in Class 4

| Mistake                                                                    | What goes wrong                                                                             |
|----------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| Compute mutual info on the FULL data                                       | Leakage.                                                                                    |
| Compute target encoding on the FULL data                                   | Leakage.                                                                                    |
| Drop columns without writing why                                           | Module 4 students will not understand.                                                      |
| Keep review as a numeric feature                                           | Crash. It is text. Keep it in the file but NOT in the model input for now.                  |
| Drop <code>n_side_effect_keywords</code> because it correlates with rating | This is NOT leakage. The feature is computed from the TEXT, not the rating itself. Keep it. |

## Self-check before Class 5

- ☐ Train/test split done FIRST.
  - ☐ Target encoding computed on TRAIN ONLY.
  - ☐ Final ~15 columns + rating + `is_effective` + review.
  - ☐ You wrote down WHY for each kept column.
  - ☐ 3+ exploratory charts.
  - ☐ 1 chart for Dr. Anya.
  - ☐ `reviews_step4.parquet` saved to Drive.
- 

## Class 5 — Pipelines

**Scenario reminder:** Dr. Anya says: "Your cleaning code is in 4 different notebooks. When a new review arrives tomorrow, you cannot copy 4 notebooks to the server. We need ONE object that does everything."

### Your goal

Put EVERY cleaning step inside ONE Pipeline object. Production-ready code. Try both REGRESSION (predict rating) and CLASSIFICATION (predict `is_effective`).

### Inputs

- `reviews_step4.parquet` (selected columns)

## Outputs

- `drug_reviews_pipeline.joblib` saved in Drive
  - 1 confusion-matrix chart + 1 ROC curve chart (classification)
  - 1 scatter plot of predicted-vs-actual ratings (regression)
  - A pipeline that takes RAW input and produces predictions
- 

## Phase A — Explore the data first

### Exploratory chart 1 — Confusion matrix (after classification training)

- After you train the classifier, build a confusion matrix.
- **HINTS:**
  - `from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay.`
  - `cm = confusion_matrix(y_test, y_pred).`
  - `ConfusionMatrixDisplay(cm, display_labels=['Not effective', 'Effective']).plot().`
- **What you learn:** How many “effective” did we CATCH? How many did we MISS?

### Exploratory chart 2 — ROC curve

- **HINTS:**
  - `from sklearn.metrics import RocCurveDisplay.`
  - `RocCurveDisplay.from_estimator(pipeline, X_test, y_test).`
- **What you learn:** The trade-off between recall and false alarms. The AUC number.

### Exploratory chart 3 — Predicted vs actual rating (regression)

- After you train a regression model:
  - **HINTS:**
    - Scatter plot: x-axis = true rating, y-axis = predicted rating.
    - Draw a diagonal line for reference.
  - **What you learn:** Where the model is correct, where it makes mistakes.
- 

## Phase B — Build the pipeline

### Step 1 — Decide which columns are numeric and which are categorical

- **HINTS:** Make two lists.
- **EXAMPLE:**
  - numeric: `usefulCount_log, review_year, review_month, review_age_years, review_length, n_sentences, n_caps_words, n_side_effect_keywords, n_positive_keywords, n_negative_keywords, has_specific_dose, mentions_doctor, mentions_pregnancy, drugName_target_encoded.`
  - categorical: `condition_top10_bucket.`

### Step 2 — Build the numeric mini-pipeline

- **WHAT:** 2 steps: imputer (fill missing) + scaler.
- **HINTS:**
  - `from sklearn.pipeline import Pipeline.`
  - `from sklearn.impute import SimpleImputer.`
  - `from sklearn.preprocessing import StandardScaler.`
  - Skeleton:

```
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='___')),
    ('scaler', StandardScaler()),
])
```

- Fill in the blank: best strategy for numeric? ('median')

### Step 3 — Build the categorical mini-pipeline

- **WHAT:** 2 steps: imputer + one-hot encoder.
- **HINTS:**
  - from sklearn.preprocessing import OneHotEncoder.
  - Skeleton:

```
categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='___')),
    ('onehot', OneHotEncoder(handle_unknown='___', sparse_output=___)),
])
```
  - Fill in blanks. (Strategy 'most\_frequent', handle\_unknown 'ignore', sparse\_output False.)
- **WHY handle\_unknown='ignore'?** In production, a new condition bucket may appear. We do not want the code to crash.

### Step 4 — Combine into a ColumnTransformer

- **HINTS:**
  - from sklearn.compose import ColumnTransformer.
  - It takes a list of tuples: (name, transformer, list\_of\_columns).

### Step 5a — Add the CLASSIFIER on top (is\_effective)

- **HINTS:**
  - from sklearn.linear\_model import LogisticRegression.
  - LogisticRegression(class\_weight='balanced', max\_iter=1000, random\_state=42).
  - Put it in a Pipeline with the preprocessor.
- **WHY class\_weight='balanced'?** About 65-70% of reviews are “effective”. Without balanced, the model just predicts “effective” for everyone and gets 65% accuracy (but useless).

### Step 5b — Add the REGRESSOR on top (rating)

- **HINTS:**
  - from sklearn.linear\_model import Ridge.
  - Ridge(alpha=1.0, random\_state=42).
  - Put it in a SECOND Pipeline with the same preprocessor.
- **WHY Ridge instead of LinearRegression?** Ridge handles correlated features better. We have several “count” features.

### Step 6 — Train and evaluate BOTH

- **HINTS:**
  - Classifier:

```
* classifier_pipeline.fit(X_train, y_train_eff).
* y_pred = classifier_pipeline.predict(X_test).
* from sklearn.metrics import classification_report.
```
  - Regressor:

```
* regressor_pipeline.fit(X_train, y_train_rating).
* y_pred_rating = regressor_pipeline.predict(X_test).
* from sklearn.metrics import mean_absolute_error, r2_score.
```
- **EXPECTED:**

- Classifier: F1 around 0.65-0.75.
- Regressor: MAE around 2-2.5 (rating is 1-10 scale).
- Both are baselines. Module 4 will improve.

## Step 7 — Save the trained pipelines

### • HINTS:

- `import joblib.`
- `joblib.dump(classifier_pipeline, '/content/drive/MyDrive/drug_reviews_lab/drug_reviews_classifier.pkl')`
- `joblib.dump(regressor_pipeline, '/content/drive/MyDrive/drug_reviews_lab/drug_reviews_regressor.pkl')`

## Phase C — Make ONE chart for Dr. Anya

### Dr. Anya's chart — "Where the model is right and where it is wrong"

Show the confusion matrix as a labeled table or heatmap. Add the numbers.

|                        | Predicted not effective | Predicted effective |
|------------------------|-------------------------|---------------------|
| Actually not effective | true negatives          | false alarms        |
| Actually effective     | missed effective        | caught effective    |

- **HINTS:** Use `seaborn.heatmap()` on the confusion matrix with `annot=True, fmt='d'`.
- **Title:** "Of 10,000 'not effective' reviews, our baseline catches ~6,800".
- **Takeaway for Dr. Anya:** "We catch about 70% of negative reviews. We sometimes flag a happy review as negative. Module 4 will tune this. Also: most errors are on borderline cases (rating 6-7), which are hard for any human too."

## Common mistakes in Class 5

| Mistake                                                               | What goes wrong                                            |
|-----------------------------------------------------------------------|------------------------------------------------------------|
| Build the pipeline AFTER you scaled the data manually                 | Pipeline scales it twice. Numbers wrong.                   |
| Forget <code>sparse_output=False</code> in <code>OneHotEncoder</code> | Some downstream code breaks.                               |
| Use <code>class_weight='balanced'</code> AND SMOTE                    | Over-correction. Pick one.                                 |
| Forget <code>random_state</code>                                      | Results change every run. Hard to debug.                   |
| Use <code>LinearRegression</code> and get negative predictions        | Ridge with simple post-processing (clip to 1-10) is safer. |

## Self-check before Class 6

- ☐ Pipeline has preprocessor + model.
- ☐ Preprocessor has numeric + categorical transformers.
- ☐ `handle_unknown='ignore'` set.
- ☐ `class_weight='balanced'` set (classifier).
- ☐ Confusion matrix + ROC curve + scatter (regression) charts.
- ☐ Both pipelines saved to Drive.

## Class 6 — End-to-End Lab (THE BIG ONE)

**Scenario reminder:** Dr. Anya is in the meeting room. She wants the FINAL clean dataset on her desk in 90 minutes. This is the lab.

## Your goal

Take 2 raw TSV files. Produce ONE final .parquet file with the exact 22-column schema. Plus a 1-page findings report.

## Time

**90 minutes** of focused work.

## Inputs

- The 2 raw TSV files in your Drive folder

## Outputs

- drug\_reviews\_clean.parquet (~215,000 rows × 22 columns) in Drive
  - findings.md (~1 page)
  - Your full Colab notebook
- 

## Phase A — Explore the data first (10 minutes)

You will reuse charts from Classes 1-5. Pick 3 of them. Put them all on ONE PAGE of your notebook with markdown headers: 1. Rating distribution (the U-shape) 2. Side-effect keyword count vs rating (your most important chart) 3. Top 10 most important features

These 3 charts tell Dr. Anya the whole story.

---

## Phase B — Build the final dataset (70 minutes)

### Required output schema

Your drug\_reviews\_clean.parquet MUST have these 22 columns, with these names and types:

| #  | Column                  | Type   | Source                                         |
|----|-------------------------|--------|------------------------------------------------|
| 1  | review_id               | int    | renamed from uniqueID                          |
| 2  | drugName                | string | raw                                            |
| 3  | drugName_target_encoded | float  | engineered (TRAIN-only mean)                   |
| 4  | condition               | string | raw, NaN filled with "Unknown"                 |
| 5  | condition_top10_bucket  | string | engineered (Mental Health, Pain, etc. + Other) |
| 6  | usefulCount             | int    | raw                                            |
| 7  | usefulCount_log         | float  | engineered (np.log1p)                          |
| 8  | review_year             | int    | engineered                                     |
| 9  | review_month            | int    | engineered                                     |
| 10 | review_age_years        | float  | engineered                                     |
| 11 | review_length           | int    | engineered                                     |
| 12 | n_sentences             | int    | engineered                                     |
| 13 | n_caps_words            | int    | engineered                                     |
| 14 | n_side_effect_keywords  | int    | engineered (rule-based count)                  |



| #  | Column              | Type      | Source                        |
|----|---------------------|-----------|-------------------------------|
| 15 | n_positive_keywords | int       | engineered (rule-based count) |
| 16 | n_negative_keywords | int       | engineered (rule-based count) |
| 17 | has_specific_dose   | int (0/1) | engineered (regex)            |
| 18 | mentions_doctor     | int (0/1) | engineered                    |
| 19 | mentions_pregnancy  | int (0/1) | engineered                    |
| 20 | is_effective        | int (0/1) | engineered (rating >= 7)      |
| 21 | rating              | int 1-10  | TARGET (raw)                  |
| 22 | review              | string    | raw, kept for Module 7        |

### Lab phases (timed)

| Phase                               | Time   | What you do                                                                         |
|-------------------------------------|--------|-------------------------------------------------------------------------------------|
| 1. Load                             | 10 min | Load both TSVs with sep='\t'. Confirm shapes.                                       |
| 2. Combine + clean                  | 15 min | pd.concat, rename, parse date, fill missing condition.                              |
| 3. Bucket conditions + encode       | 10 min | Map top 10 to named groups. Add "Other".                                            |
| 4. Train/test split BEFORE encoding | 5 min  | So target encoding uses train only.                                                 |
| 5. Engineered features              | 25 min | usefulCount_log, date parts, text-shape, keyword counts, regex flags, is_effective. |
| 6. Validate schema + save           | 10 min | Check all 22 columns. Save .parquet.                                                |
| 7. Findings                         | 15 min | Write findings.md.                                                                  |

**Total: 90 minutes.**

### Validation checklist (run before saving)

```

expected_cols = [
    'review_id', 'drugName', 'drugName_target_encoded',
    'condition', 'condition_top10_bucket',
    'usefulCount', 'usefulCount_log',
    'review_year', 'review_month', 'review_age_years',
    'review_length', 'n_sentences', 'n_caps_words',
    'n_side_effect_keywords', 'n_positive_keywords', 'n_negative_keywords',
    'has_specific_dose', 'mentions_doctor', 'mentions_pregnancy',
    'is_effective', 'rating', 'review',
]
assert sorted(df.columns.tolist()) == sorted(expected_cols), "Schema mismatch!"
assert df.shape[0] > 200_000, "Too few rows!"
assert df['rating'].between(1, 10).all(), "Rating out of range!"
assert df['is_effective'].isin([0, 1]).all(), "is_effective not binary!"
print("Schema OK. Rows:", df.shape[0])

```

### Phase C — Findings report for Dr. Anya (15 minutes)

Write findings.md with these sections:

## Final numbers

- Final row count: \_\_\_\_\_
- `is_effective` rate: \_\_\_\_\_% (should be 65-75%)
- Number of unique drugs: \_\_\_\_\_
- Number of named condition buckets: \_\_\_\_\_

## Top 3 insights

1. \_\_\_\_\_
2. \_\_\_\_\_
3. \_\_\_\_\_

(Example insight: “Reviews with 5+ side-effect words have an average rating of 3.2, vs 8.1 for reviews with zero side-effect words.”)

## One question to investigate in Module 4

- \_\_\_\_\_

## One ethical concern

- \_\_\_\_\_ (Example: “Some reviews mention pregnancy and specific doses. We must NEVER use the model to make medical decisions. The model is for research only.”)

## One chart that summarizes everything

Embed your most important chart (the side-effect keywords vs rating one).

\_\_\_\_\_

## Grading rubric (100 points)

| Item                                                          | Points    |
|---------------------------------------------------------------|-----------|
| All 22 columns exist with the right names and dtypes          | 25        |
| Cleaning decisions written in markdown                        | 10        |
| Pipeline is reproducible (re-run from 2 TSVs in ONE command)  | 15        |
| TSV loaded with <code>sep='\t'</code> (no single-column bug)  | 5         |
| Target encoding uses TRAIN ONLY (no leakage)                  | 5         |
| <code>is_effective</code> rate is between 65-75%              | 10        |
| <b>At least 3 exploratory + 1 explanatory chart per class</b> | <b>15</b> |
| <code>findings.md</code> has insights AND an ethical note     | 10        |
| Code is clean (comments, sensible variable names)             | 5         |

## Submit

Upload to `module-3/class_6/submissions/<YourTeamName>/`: - `drug_reviews_clean.parquet` - Your Colab notebook (File > Download > Download .ipynb) - `findings.md`

\_\_\_\_\_

## Tips for the whole module

1. **Do NOT copy-paste code from this guide.** This guide gives you HINTS only. Write the code yourself. You will learn much faster.
2. **Save your notebook to Drive often.** Colab disconnects after 12 hours.
3. **Save intermediate .parquet files to Drive** (reviews\_step1, reviews\_step2, etc.). If something breaks, you do not redo everything.
4. **Pair-program.** One student types, one reads. Switch every 20 minutes.
5. **Make a chart BEFORE you write cleaning code.** You must SEE the problem before you can fix it.
6. **Make a chart AFTER each step.** Confirm your code did what you expected.
7. **Ask the mentor early.** If you are stuck for 20 minutes, ask. Do not waste an afternoon.
8. **Respect the patient data.** Do not screenshot full reviews. Do not paste reviews into chat apps. Treat them as private even though they are technically public.
9. **Remember the medical context.** This is not e-commerce. Wrong answers here could affect health. Always include an ethical note in your findings.

Good luck.